

Introduction to Assembly

- Introduction to Assembly
 - Introduction
 - Learning objectives
 - Assembling and loading an assembly program
 - Building the project
 - **Task 1:** Assembling by hand
 - **Task 2:** Sections
 - Memory content on the target system after loading
 - Code section
 - **Task 3:** Linker tasks
 - **Task 4:** Deviations
 - Data section (READ ONLY)
 - **Task 5:** Memory view
 - Data section (READ & WRITE)
 - **Task 6:** Function of the program
 - **Task 7:** Verifying the table
 - Grading

Introduction

In this lab you will work with assembly for the first time. You will learn the possibilities of the remote debugger to visualise and change the memory and register content. You will trace the influence of several data transfer commands and their addressing modes with the remote debugger.

The file `<ct-students-path>/instruction-reference/instruction-reference.pdf` lists and explains the most important instructions (of the thumb instruction set) and is rather handy for all future laboratories.

It is also recommended to bring this document to all exams.

Learning objectives

- You can assemble, link, upload and execute an assembly program on the target hardware.
- You know how to use the remote debugger, to visualise and change the content of memory, registers and ports.
- You understand the different addressing modes in simple programs and are able to apply them.

Assembling and loading an assembly program

In this chapter you'll learn how to assemble and link a program for the CT Board and how to upload it to the target hardware.

Assembling means to translate the text based source code (coded in assembly language) into op codes used by the target hardware. This process is done by an assembler. The reverse process, translating op code to assembly language, is called disassembling.

Building the project

To build the project run `make`, or `make rebuild` when working in the terminal. When working with VS Code make sure to add only the project folder to your workspace (not a parent directory), then press `CTRL + SHIFT + P` and run the task `build`, or `rebuild`.

Task 1: Assembling by hand

Which operation codes are generated by the assembler for the following assembly instructions? Search for the corresponding op code in the list file and use the disassembly table to decode the hexadecimal values. Fill in the gaps in the following table based on the example.

Ignore the dots, they serve to keep the spacing during document conversion.

Assembly Code	Operation Code
Example	0x21FE
<code>movs r1, #0xFE</code>	00100 001 11111110 ----- <code>movs. r1. imm8....</code>
<code>movs r2, #MY_CONST</code>	

Assembly Code	Operation Code
<code>mov r11, r2</code>	
<code>ldr r0, [r7]</code>	
<code>str r3, [r7, r6]</code>	

Task 2: Sections

The given program (`./src/transbf.s`) is split into three sections. What are the three sections and what are their properties?

How many bytes does each section contain?

After assembling, each section begins at the address `0x0000_0000`. The physical addresses are assigned during the linking process.

Uploading onto the Target Hardware

Switch on the target hardware. Ensure that the USB connection on the left side of the target hardware is connected to the host computer.

Caution: Don't press "Run (F5)", the program has to be halted for the following manipulations.

Memory content on the target system after loading

While the `.lst` files contain the listing of the assembly translation of each module, the project's `.map` file contains the listing of the linker actions and symbol resolution in the final executable.

Code section

Before we run the program we want to take a look at the memory content on the target hardware. We want to see where exactly the program has been loaded.

For this we need a way to inspect the memory contents. See the section(s) on debugging in the rootlevel readme.

When using GDB the following command serves as example: `x/10xh`

When using VS Code remember, that the register view only updates **on** code execution halt, not **while** executions is halted.

Task 3: Linker tasks

Open the file `bin/transbf.map` file in the text editor of your choice and search for the symbol `my_code`. The associated address needs to be rounded to an even value.

In **VS Code**, after starting the debugger, select the debug tab and add a new view.

In **Keil** click on Memory 1 and enter the start address of the main function code section in the field 'Address'.

Can you locate the operation codes from the list file in memory?

Load Address

By default the debugger loads the program at address `0x0800_0000`. At this address the flash memory is located on the target hardware. The memory space from here to the beginning of the main function contains the initialisation code for the microcontroller.

Task 4: Deviations

Compared to the instruction reference the order of the two bytes of machine code is reversed. What could be the reason?

Some bytes will be defined after the creation of the list file (by the linker). At these positions the memory content differs from the list file. Find corresponding bytes! What could be the reason?

Data section (READ ONLY)

The sample program defines several global variables in the data section `my_const`. The assembler directives `.word`, `.hword` and `.byte` reserve memory. An assembler directive is a directive of the programmer for the translation program, the assembler. The assembler directive does not get translated into executable operation codes, i.e. there is no corresponding op code.

Task 5: Memory view

The debugger will allocate the storage region of the read-only data section right after the read only code section. The starting address of this section is given by the first constant in the read only data section. I.e. the first constant is at label `addr_dip_switch`. Search the map file (`bin/transbf.map`) for the start address of the respective memory section and inspect the memory with the debugger. Fill in the following table with the values and start addresses of the constants.

Variable name	Content	Start address
<code>addr_dip_switch</code>		
<code>const_table[0]</code>		
<code>const_table[1]</code>		
<code>const_table[2]</code>		
<code>const_table[3]</code>		

Data section (READ & WRITE)

The sample program defines a global variable in the read/write data section `my_var`. The assembler directive `.space` reserves memory space and fills it with zero.

The data sections are mapped to the physical RAM of the CT board starting at `0x2000_0000`. The stack is located after our custom data sections `my_var` and `my_const`. You find the respective information in the `bin/transbf.map` file by searching for the section `my_var`.

Task 6: Function of the program

The given program `src/transbf.s` demonstrates different commands, used to load and store values. It shows how constants are defined and loaded, and how the load and store commands are used.

Study the code in the list file. What are the results of the indicated instructions? Fill in the following table with the expected values of the target registers after the corresponding line of code has been executed.

Line	Instruction	Content of target register
*** A1 ***	<code>movs r1, #0xFE</code>	
*** A2 ***	<code>mov r11, r2</code>	
*** A3 ***	<code>ldr r3, = ADDR_DIP_SWITCH_31_0</code>	
*** A5 ***	<code>ldr r7, =addr_dip_switch</code>	
*** A6 ***	<code>ldr r1, [r7, #4]</code>	
*** A7 ***	<code>ldr r3, [r7, r6]</code>	

Task 7: Verifying the table

Execute the program step by step. Check your values in the table. Be aware of the differences between the two lines marked with A4 and A5 (LDR as literal and as pseudo instruction). Do they meet your expectations?

Grading

Criteria	Weight
The tables are filled in correctly and the questions are answered. You can explain your reasoning.	3 / 4
You briefly explain how you used the debugger and can answer questions about it.	1 / 4